(Materials of this lecture are NOT included in the midterm and final exams)

CSC3100: Red-Black Tree

Tianshu Yu

2024 Fall

Red-Black tree

- Recall binary search tree (BST)
  - Key values in the left subtree <= the node value
  - Key values in the right subtree >= the node value
- Operations:
  - insertion, deletion
  - Search, maximum, minimum, successor, predecessor.
  - O(h), h is the height of the tree.

Red-Black tree

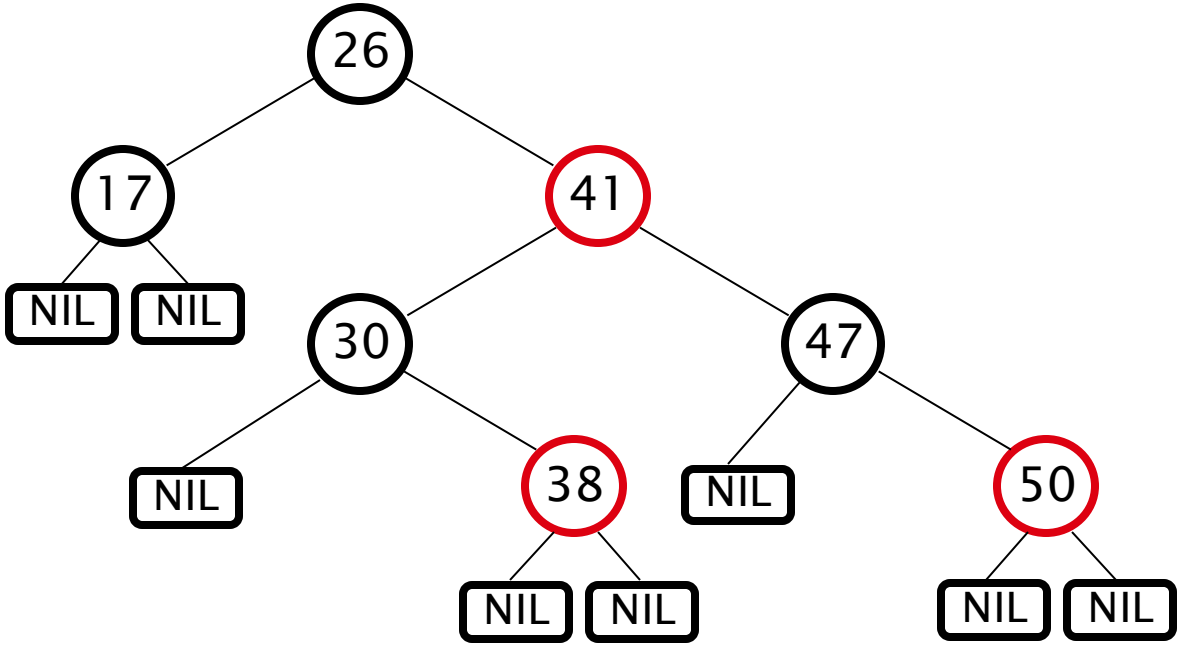
- Red-black-tree
  - Binary search tree with an additional attribute for its nodes: *color* which can be **red** or **black**
  - Restrict the way nodes can be colored on any path from the root to a leaf
  - Ensures that no path is more than twice as long as any other path

- "Balanced" binary search trees guarantee an $O(\log n)$ running time

Red-Black tree properties

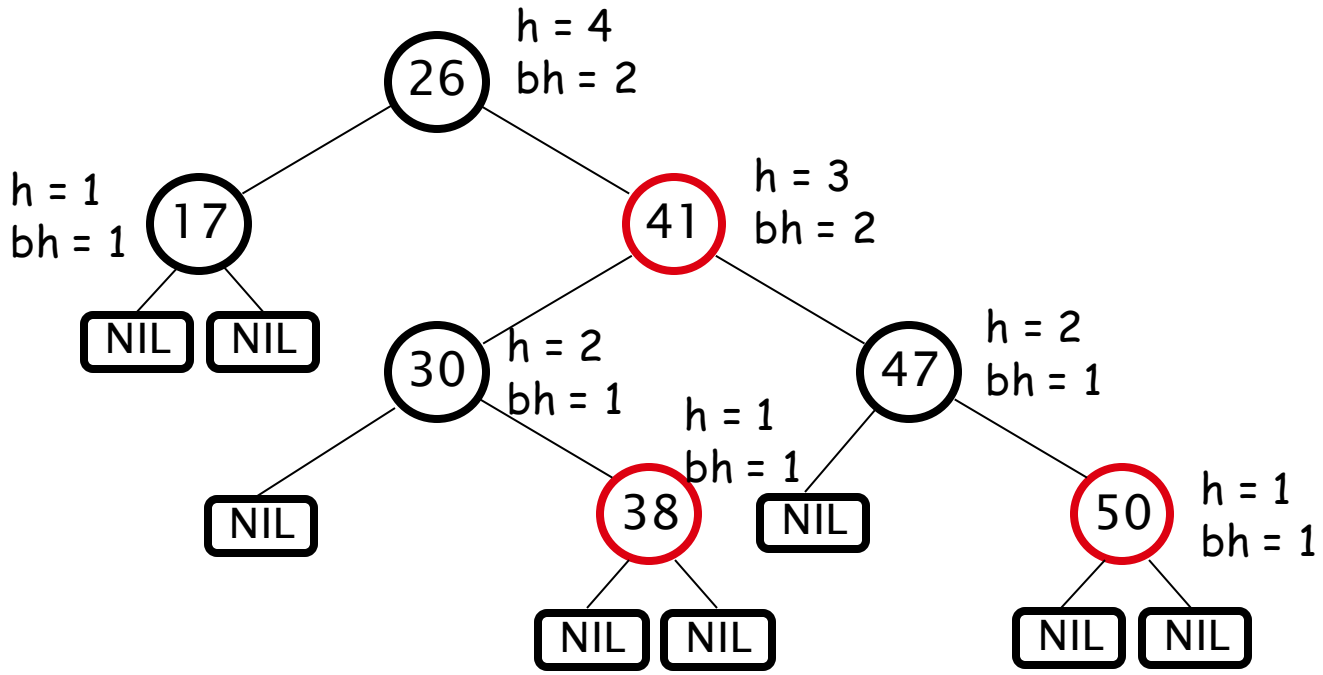
1. Definition: RBT is a BST, satisfying
   I. Every node is either red or black
   II. The root is black
   III. Every leaf (NIL) is black
   IV. If a node is red, then both its children are black. No two consecutive red nodes on a simple path from the root to a leaf
   V. For each node, all paths from that node to descendant leaves contain the same number of black nodes
2. Purpose: keep the tree balanced
3. Fields: Left, right, parent, color, key/value

Example



- For convenience we use a sentinel NIL[T] to represent all the NIL nodes at the leafs
  - NIL[T] has the same fields as an ordinary node
  - Color[NIL[T]] = BLACK
  - The other fields may be set to arbitrary values

(Black) Height of a node



- **Height of a node:** the number of edges in the **longest** path from the current node to a leaf
- **Black-height** of a node x: bh(x) is the number of black nodes (including NIL) on the path from x to a leaf, **not counting x**

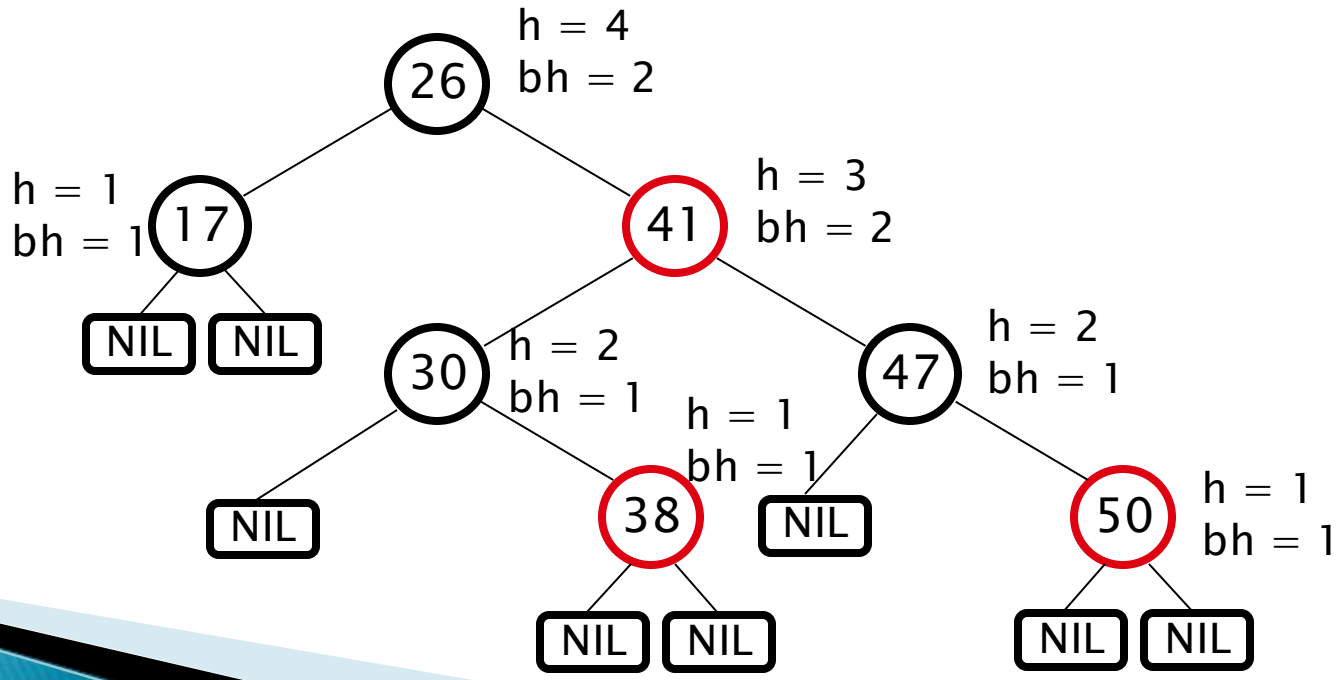
Important property of Red-Black tree

A red-black tree with n internal nodes has height <u>at most</u> $2\log(n + 1)$

- Only need to be aware of this property for this course
- If you want the proof, you might need to prove two claims first …

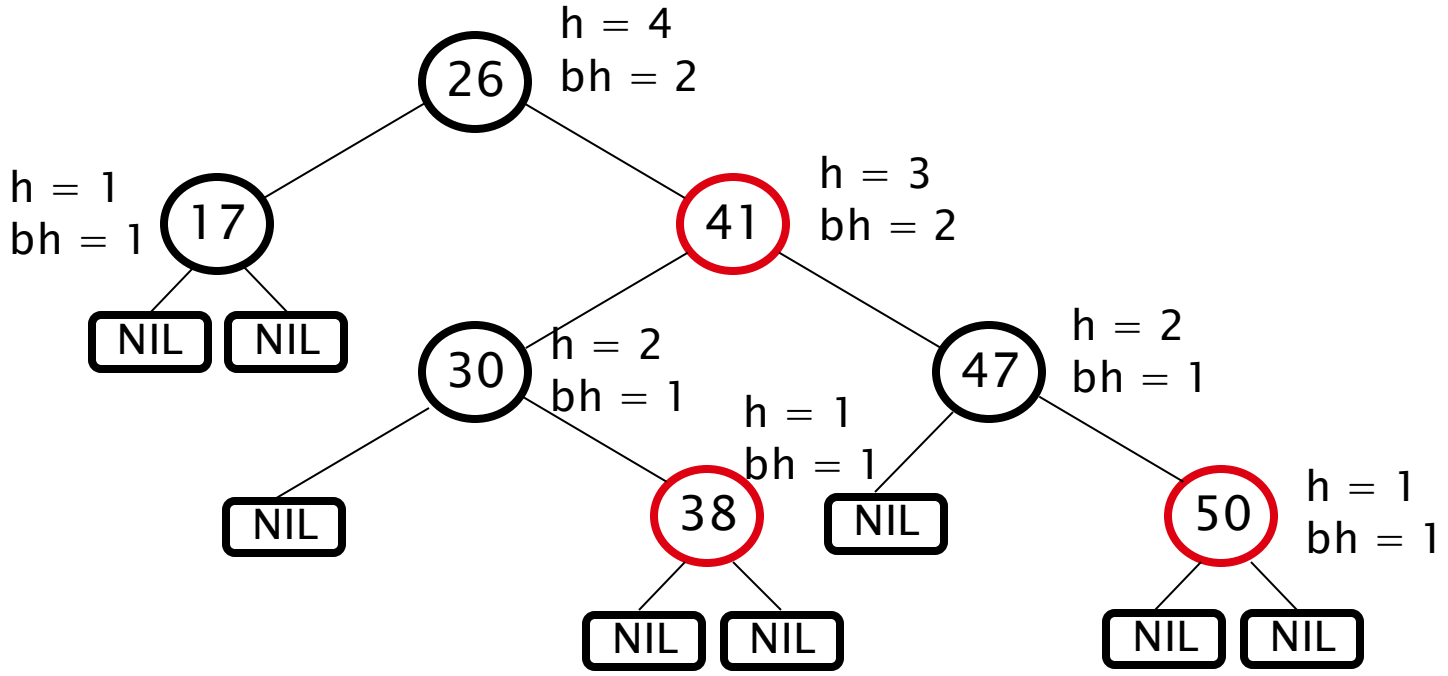
Claim 1

- Any node x with height h(x) has bh(x) ≥ h(x)/2
- **Proof**
  - By property 4, at most h/2 **red** nodes on the path from the node to a leaf
  - Hence at least h/2 are **black**



Claim 2

- The subtree rooted at any node x contains **at least** $2^{bh(x)} - 1$ internal nodes



Claim 2 (Cont'd)

Proof: By induction on **h[x]**

Basis: $h[x] = 0 \Rightarrow$

x is a leaf (=NIL[T]) $\Rightarrow$

$bh(x) = 0 \Rightarrow$

of internal nodes: $2^0 - 1 = 0$

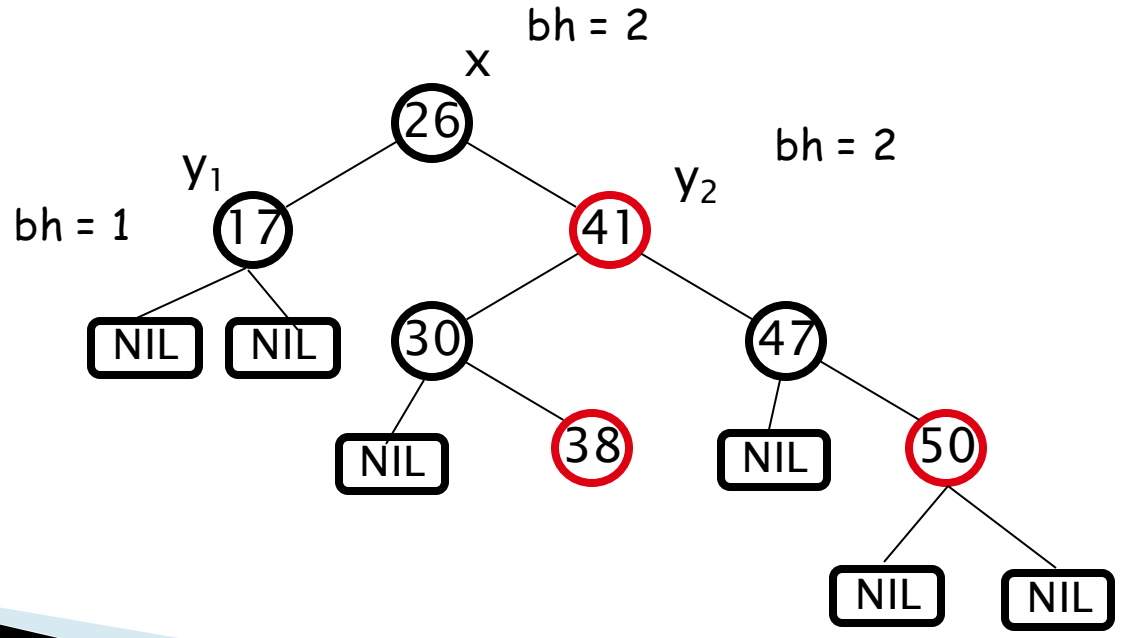
Inductive Hypothesis: assume it is true for

$h[x]=h-1$

Claim 2 (Cont'd)

Inductive step:

- Prove it for h[x]=h
- Let bh(x) = b, then any child y of x has:
  - bh (y) = b (if the child is **red**), or
  - bh (y) = b - 1 (if the child is **black**)



Claim 2 (Cont'd)

- Using inductive hypothesis, the number of internal nodes for each child of x is at least (if it is black):

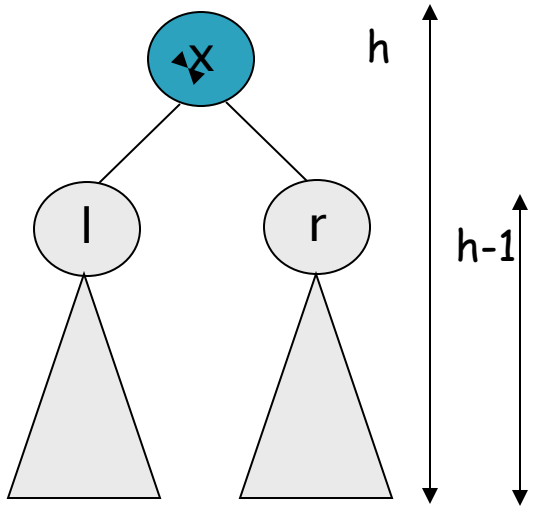
$$2^{bh(x)-1} - 1$$

- The subtree rooted at x contains at least:

$$(2^{bh(x)-1} - 1) + (2^{bh(x)-1} - 1) + 1 =$$

$$2 \cdot (2^{bh(x)-1} - 1) + 1 =$$

$2^{bh(x)} - 1$ internal nodes



$bh(l) \geq bh(x)-1$

$bh(r) \geq bh(x)-1$

Important property of Red-Black tree

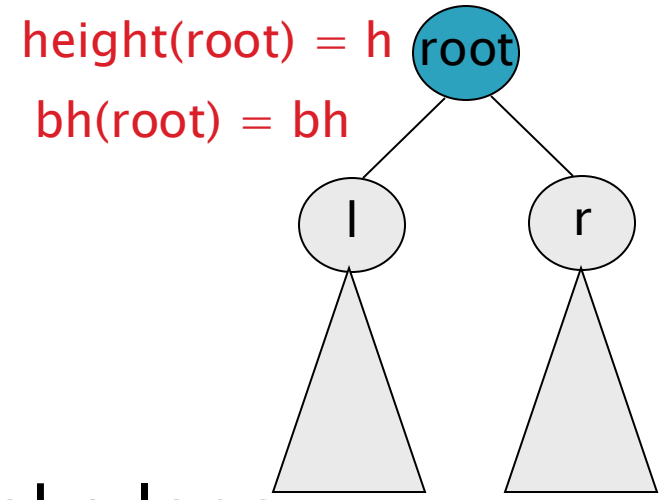
A red-black tree with n internal nodes has height <u>at most</u> $2\log(n + 1)$

Proof in the next slides.

- Claim 1: Any node x with height h(x) has $bh(x) \geq h(x)/2$
- Claim 2: The subtree rooted at any node x contains **at least** $2^{bh(x)} - 1$ internal nodes

Heights of Red-Black tree

Lemma: A red-black tree with n internal nodes has height at most $2\log(n + 1)$.



Proof:

$$n \geq 2^{bh} - 1 \geq 2^{h/2} - 1$$

number **n** of internal nodes

since $bh \geq h/2$

- Add 1 to both sides and then take logs:

$$n + 1 \geq 2^{bh} \geq 2^{h/2}$$

$$\log(n + 1) \geq h/2 \Rightarrow$$

$$h \leq 2 \log(n + 1)$$

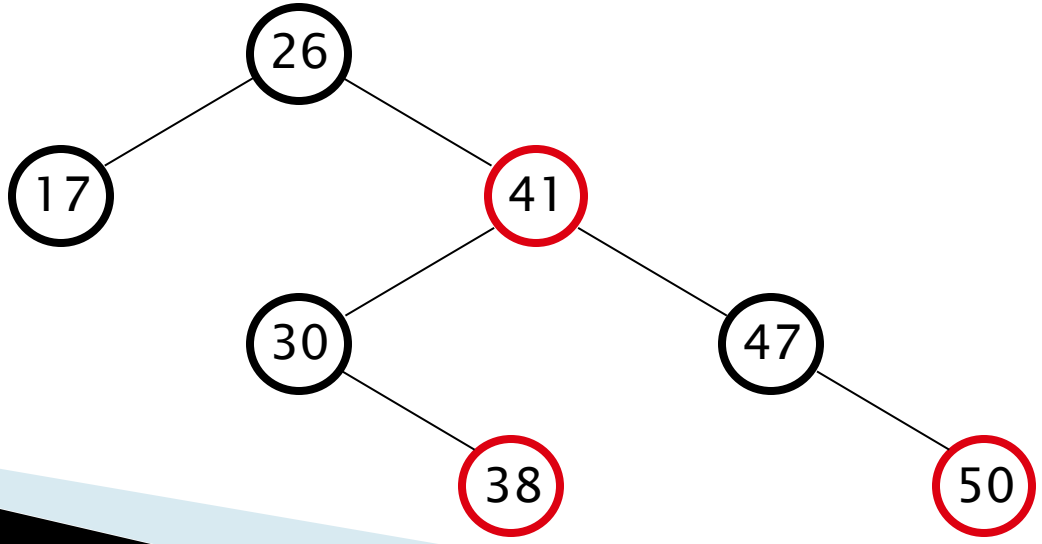
Operations of Red-black tree

- The non-modifying operations: MINIMUM, MAXIMUM, and SEARCH run in $O(h)$ time
  - They take $O(\log n)$ time on red-black trees
- What about INSERT and DELETE?
  - They will still run on $O(\log n)$
  - We have to guarantee that the modified tree will still be a red-black tree

INSERT operation

INSERT: Suppose we want to insert 35. What color to make the new node?

- Red?
  - Property 4 is violated: if a node is red, then both its children are black
- Black?
  - Property 5 is violated: all paths from a node to its leaves contain the same number of black nodes

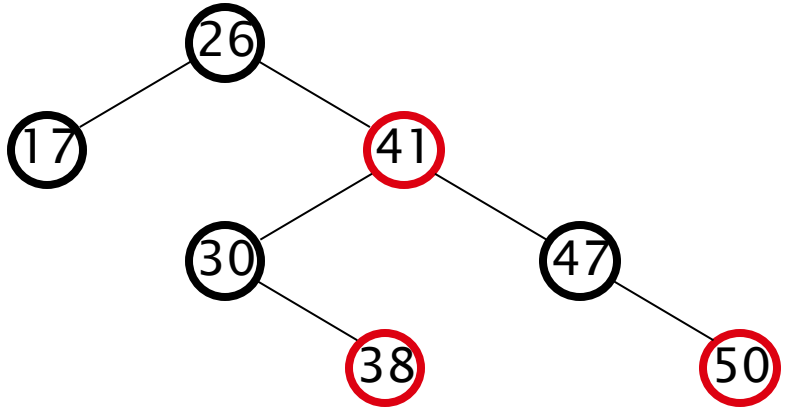


Rotations

- After insertion and deletion on red-black trees, we need to restore the red-black tree properties

- Rotations take a red-black tree and a node within the tree and:
  - Two types of rotations: Left & right rotations
  - Together with some node re-coloring they help restore the red-black tree property
  - Change some of the pointer structure
  - Do not change the binary search tree property

DELETE operation

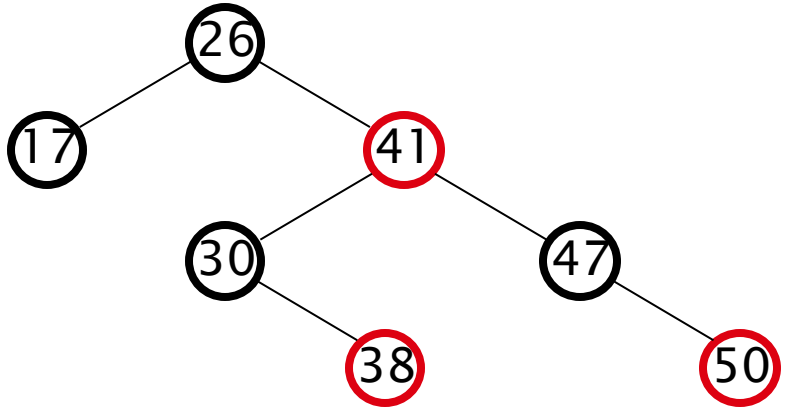


DELETE: the color of the node to be removed -- **red**

1. Every node is either **red** or **black** OK!
2. The root is **black** OK!
3. Every leaf (NIL) is **black** OK!
4. If a node is red, then both its children are black OK!
5. For each node, all paths from the node to descendant leaves contain the same number of black nodes OK!

Note: the deletion of a red node is the same as the deletion of a node in BST

DELETE operation



DELETE: the color of the node to be removed -- **Black**

1. Every node is either **red** or **black** OK!
2. The root is **black**
3. Every leaf (NIL) is **black** OK!
4. If a node is red, then both its children are black

Not OK! If removing the root and the child that replaces it is **red**

Not OK! Could create two red nodes in a row

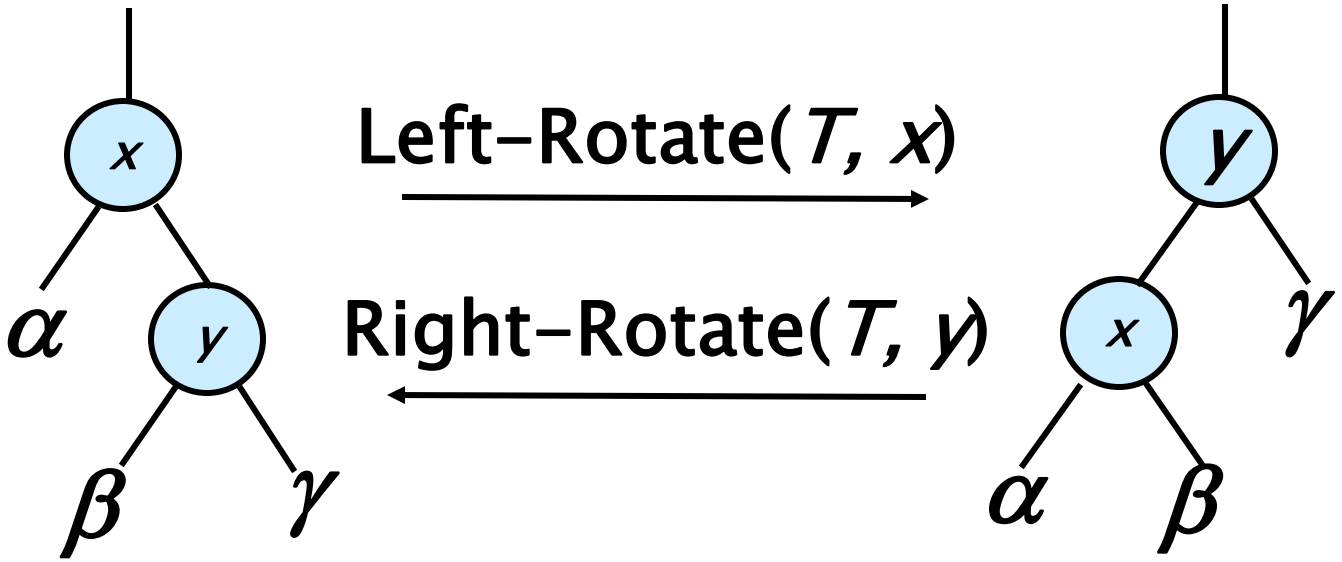
Not OK! Could change the black heights of some nodes

5. For each node, all paths from the node to descendant leaves contain the same number of black nodes

Rotations

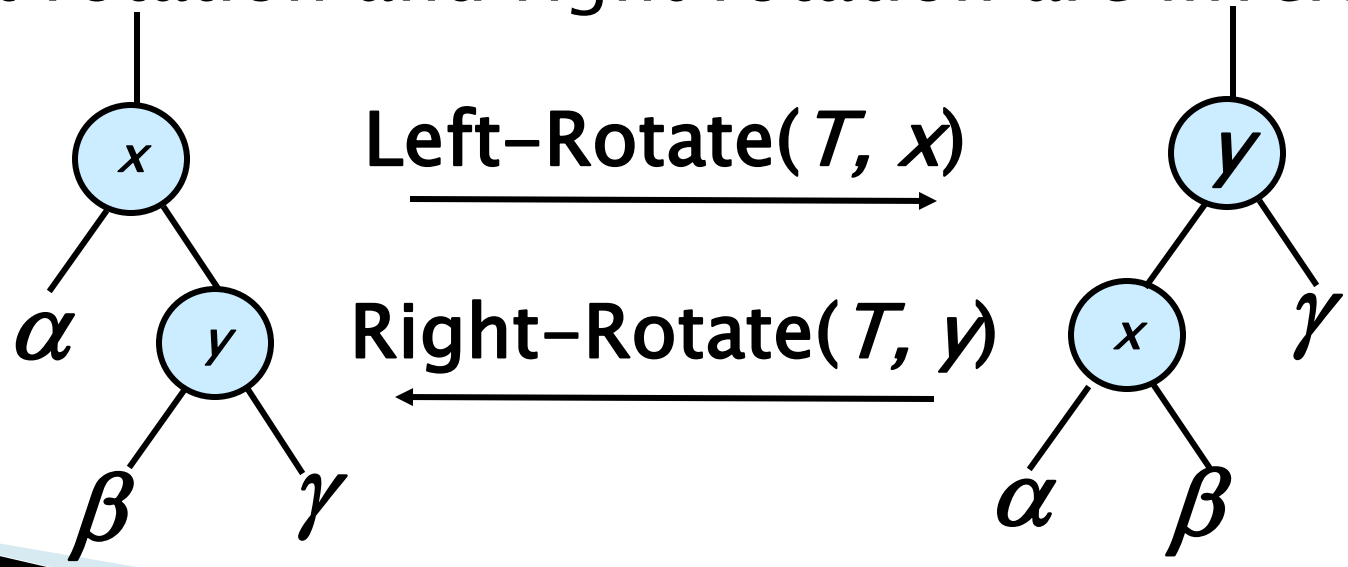
- Operations for re-structuring the tree after insert and delete operations on red-black trees
- Rotations take a red-black-tree and a node within the tree and:
  - Together with some node re-coloring they help restore the red-black-tree property
  - Change some of the pointer structure
  - **Do not** change the binary-search tree property
- Two types of rotations:
  - Left & right rotations

Rotations



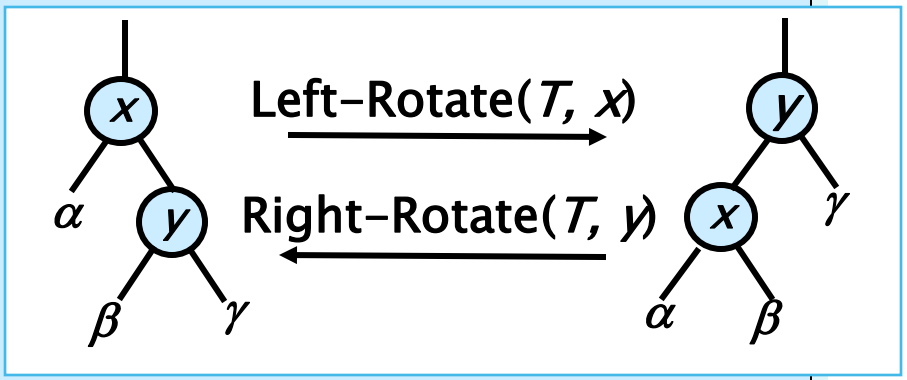
Rotations

- Rotations are the basic tree-restructuring operation for almost all *balanced* search trees.
- Rotation takes a red-black-tree and a node,
- Changes pointers to change the local structure, and
- Won't violate the binary-search-tree property.
- Left rotation and right rotation are inverses.



Left-Rotate (*T*, *x*)

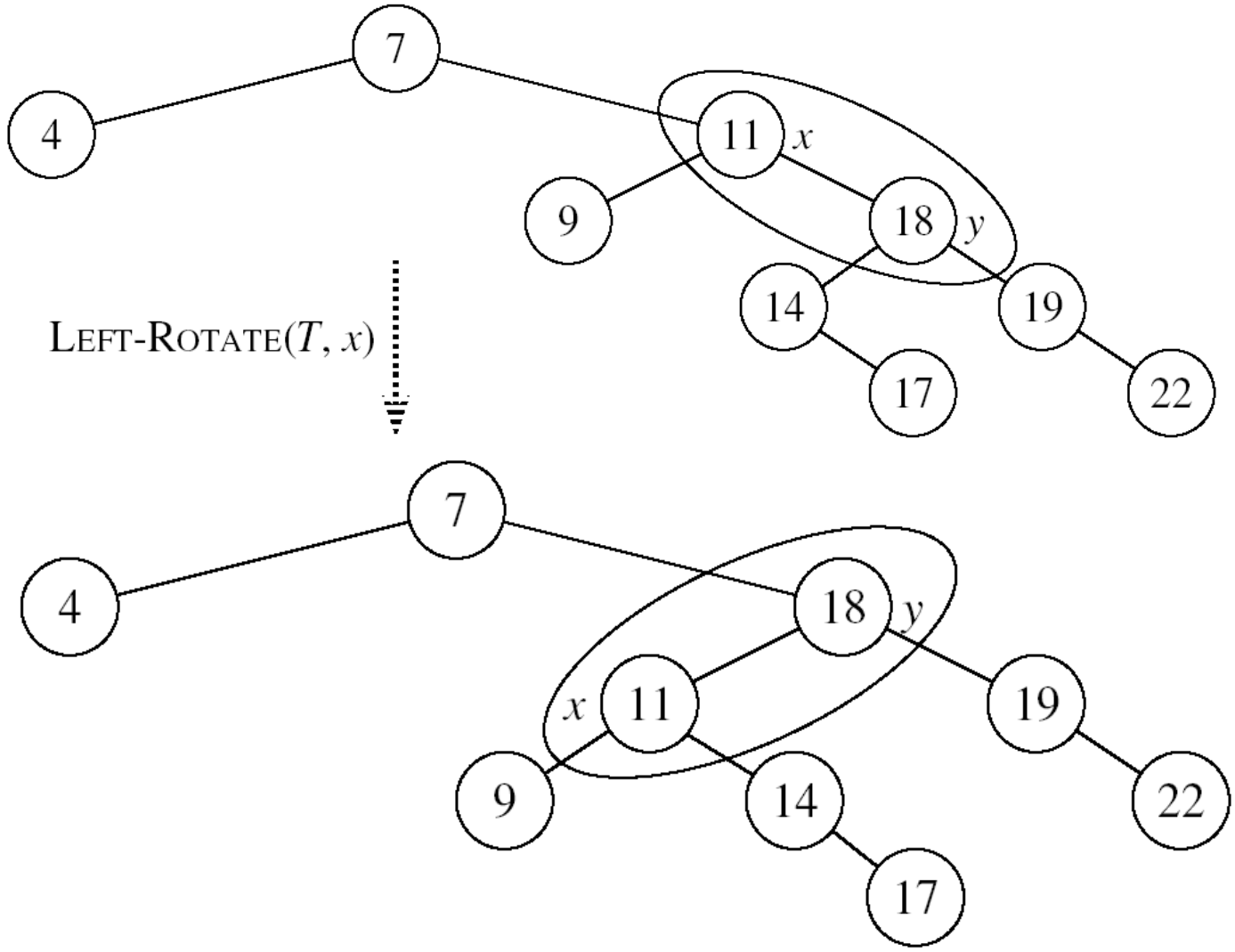
1. $y \leftarrow right[x]$ // Set y.
2. $right[x] \leftarrow left[y]$ //Turn y's left subtree into x's right subtree.
3. **if** $left[y] \neq nil[T]$
4. **then** $p[left[y]] \leftarrow x$
5. $p[y] \leftarrow p[x]$ // Link x's parent to y.
6. **if** $p[x] = nil[T]$
7. **then** $root[T] \leftarrow y$
8. **else if** $x = left[p[x]]$
9. **then** $left[p[x]] \leftarrow y$
10. **else** $right[p[x]] \leftarrow y$
11. $left[y] \leftarrow x$ // Put x on y's left.
12. $p[x] \leftarrow y$



Rotation

- The pseudo-code for Left-Rotate assumes that
  - $right[x] \neq nil[T]$, and
  - root's parent is $nil[T]$.
- Left Rotation on x, makes x the left child of y, and the left subtree of y into the right subtree of x.
- Pseudocode for Right-Rotate is symmetric: exchange *left* and *right* everywhere.
- *Time:* $O(1)$ for both Left-Rotate and Right-Rotate, since a constant number of pointers are modified.

Example



INSERT

- Goal:
  - Insert a new node z into a red-black-tree
- Idea:
  - Insert node z into the tree as for an ordinary binary search tree
  - Color the node **red**
  - Restore the red-black-tree properties
    - Use an auxiliary procedure RB-INSERT-FIXUP

INSERT in RB Trees

- Insertion must preserve all red-black properties.
- Should an inserted node be colored Red? Black?
- Basic steps:
  - Use Tree-Insert from BST (slightly modified) to insert a node x into T.
    - Procedure **RB-Insert(T, x)**.
  - Color the node x red.
  - Fix the modified tree by re-coloring nodes and performing rotation to preserve RB tree property.
    - Procedure **RB-Insert-Fixup**.

Properties affected by INSERT

1. Every node is either **red** or **black**
2. The root is **black**
3. Every leaf (NIL) is **black**
4. If a node is red, then both its children are black
5. For each node, all paths from the node to descendant leaves contain the same number of black nodes

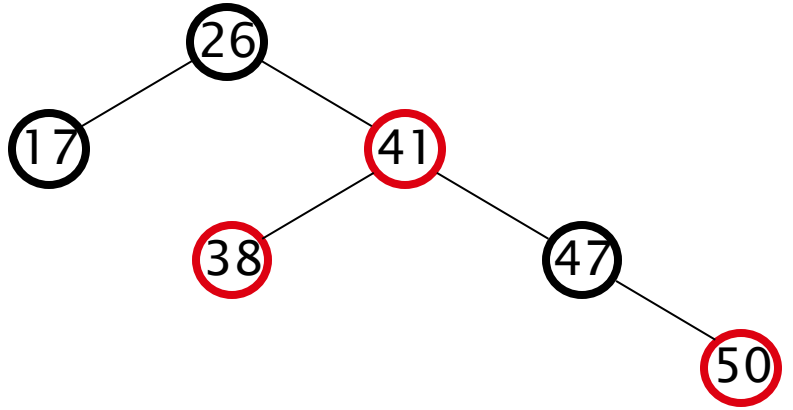
OK!

If the root is changed
⇒ May not OK

OK!

If p(z) is red ⇒ not OK
z and p(z) are both red

OK!



INSERT(T, z)

```
1.  y ← NIL
2.  x ← root[T]
3.  while x ≠ NIL
4.        do y ← x
5.           if key[z] < key[x]
6.              then x ← left[x]
7.              else x ← right[x]
8.  p[z] ← y
9.  if y = NIL
10.    then root[T] ← z
11.    else if key[z] < key[y]
12.           then left[y] ← z
13.           else right[y] ← z
14. left[z] ← NIL
15. right[z] ← NIL
16. color[z] ← RED
17. RB-INSERT-FIXUP(T, z)
```

- Initialize nodes x and y
- Throughout the algorithm y points to the parent of x

- Go down the tree until reaching a leaf
- At that point y is the parent of the node to be inserted

Sets the parent of z to be y

The tree was empty: set the new node to be the root

Otherwise, set z to be the left or right child of y, depending on whether the inserted node is smaller or larger than y's key

Set the fields of the newly added node

Fix any inconsistencies that could have been introduced by adding this new red node

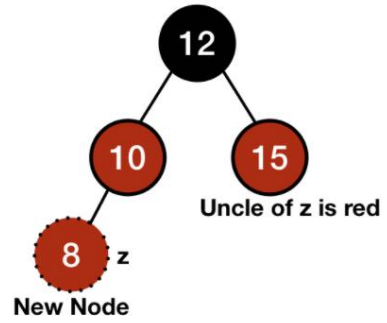
INSERT – Case 1

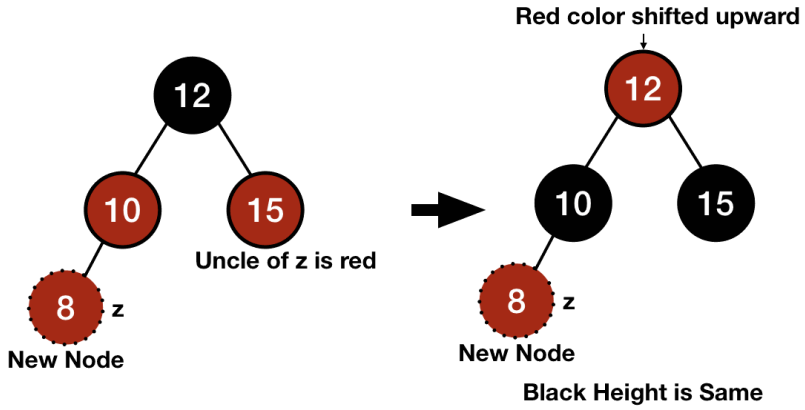
z's "uncle" (y) is **red**

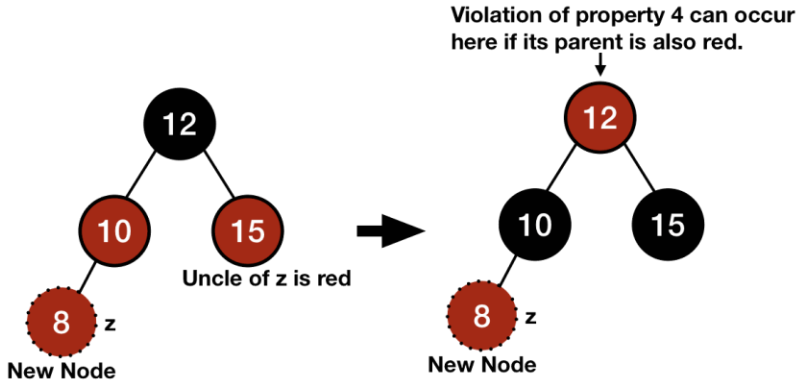
z is a left child (right is symmetric)

- p[p[z]] (z's grandparent) must be black: p[z] is red

- color p[z] ← black
- color y ← black
- color p[p[z]] ← red
- z = p[p[z]]
  - Push the **"red"** violation up the tree







INSERT – Case 2

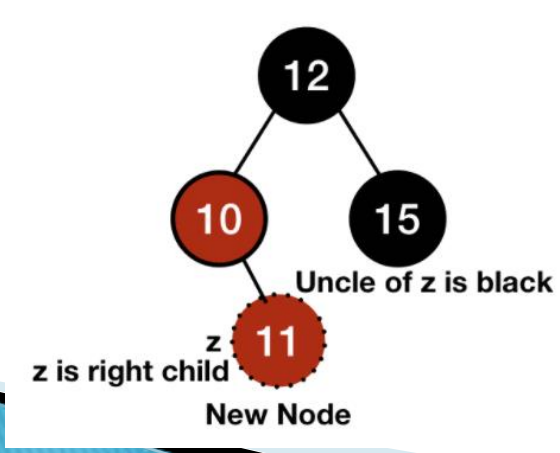
Case 2:

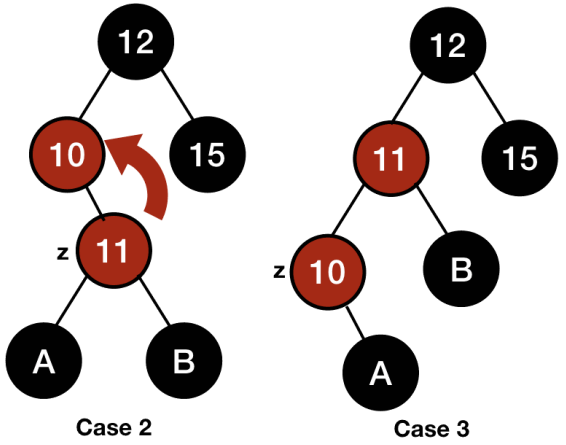
- z's "uncle" (y) is **black**
- z is a right child

Idea:

- $z \leftarrow p[z]$
- LEFT-ROTATE(T, z)

$\Rightarrow$ now z is a left child, and both z and p[z] are red $\Rightarrow$ case 3





INSERT – Case 3

Case 3:

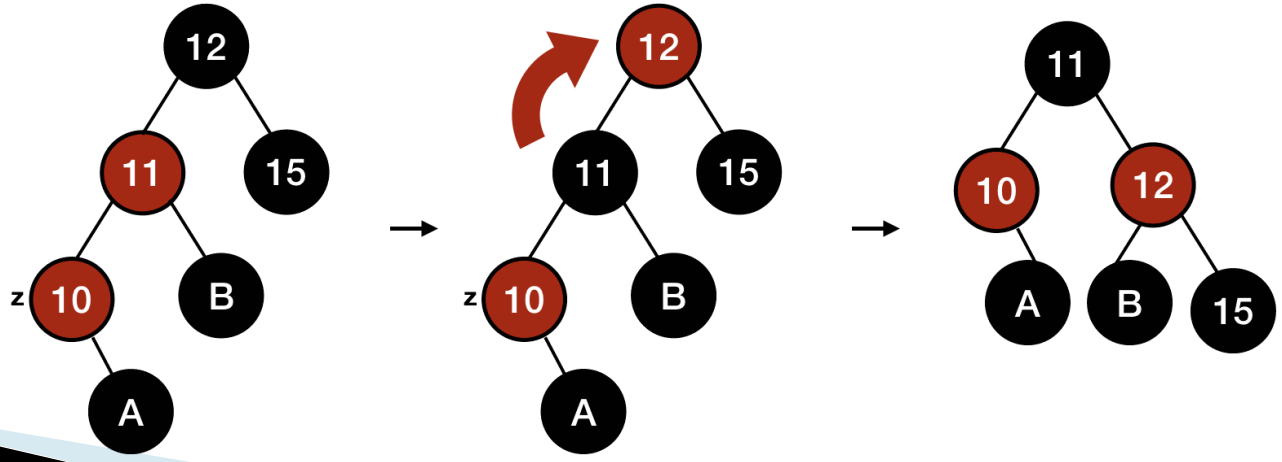
- z's "uncle" (y) is **black**
- z is a left child

NOTE: the root of this subtree becomes black!

Idea:

- color p[z] ← **black**
- color p[p[z]] ← **red**
- RIGHT-ROTATE(T, p[p[z]])

- No longer have 2 reds in a row
- p[z] is now black



INSERT_FIXUP(T,z) implementation

```
1.  while color[p[z]] = RED do
2.      if p[z] = left[p[p[z]]] then
3.          y ← right[p[p[z]]]
4.          if color[y] = RED then
5.              color[p[z]] ← BLACK
6.              color[y] ← BLACK
7.              color[p[p[z]]] ← RED
8.              z ← p[p[z]]
9.          else if z = right[p[z]] then
10.             z ← p[z]
11.             Left-rotate(T,z)
12.         color[p[z]] ← BLACK
13.         color[p[p[z]]] ← RED
14.         Right-rotate(T,p[p[z]])
15.     else (same as then clause with "right" and "left" exchanged)
16. color[root[T]] ← BLACK
```

The while loop repeats only when case1 is executed: $O(\log n)$ times

Set the value of x's "uncle"

Case 1

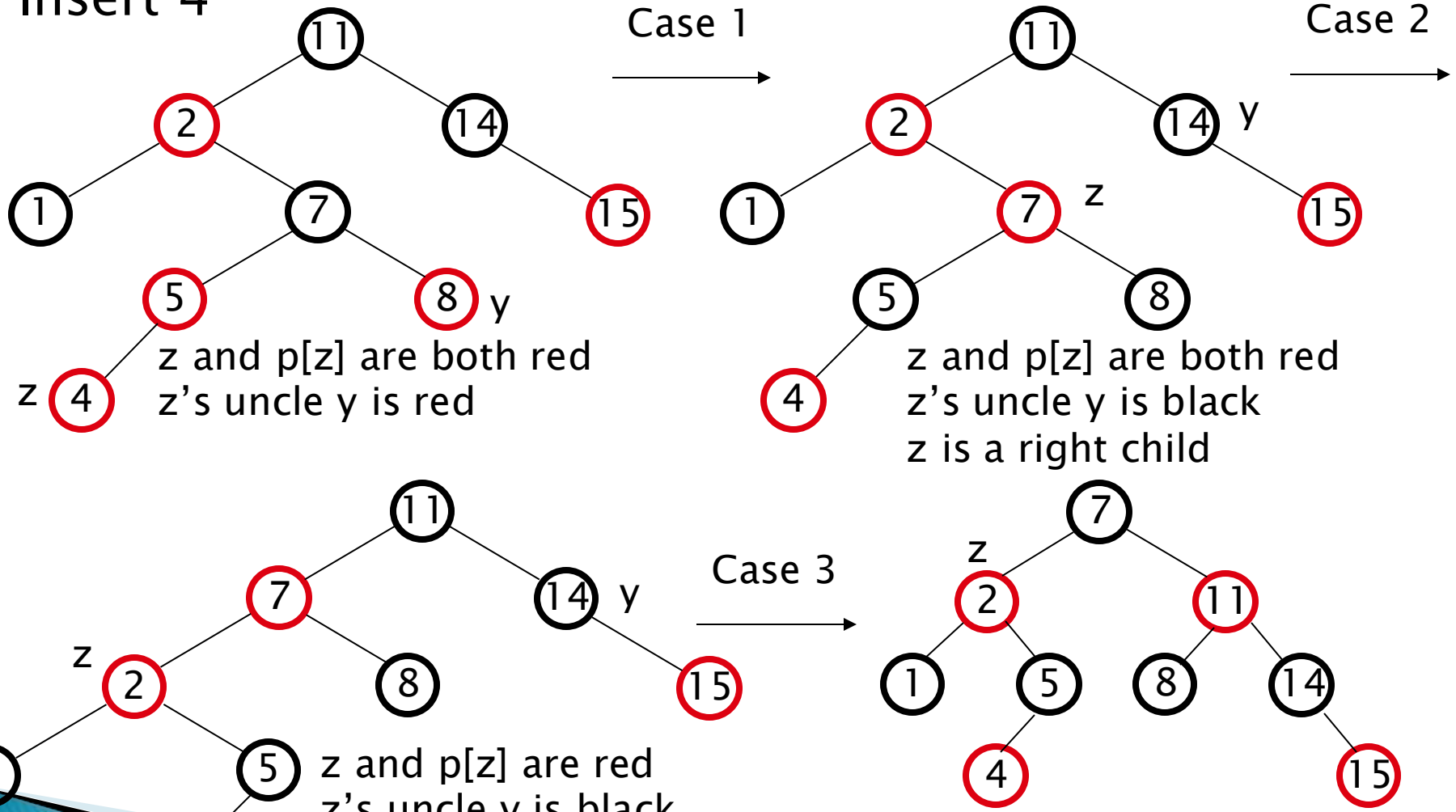
Case 2

Case 3

We may just insert the root, or the red violation may reach the root.

Example

Insert 4



Complexity analysis

- Time complexity of detailed steps
  - A red-black tree has $O(\log n)$ height
  - Search for insertion location takes $O(\log n)$ time
  - Addition to the node takes $O(1)$ time
  - The while loop will be executed at most $O(\log n)$ time
    - Each recoloring and each rotation take $O(1)$ time
    - Never performs more than two rotations, since the loop terminates if case 2 or case 3 is executed
  - Hence, an insertion in a red-black tree takes $O(\log n)$ time

Exercise 1

- When we insert a node into a red-black tree, we initially set the color of the new node to red. Why didn't we choose to set the color to black?

Deletion

- Deletion, like insertion, should preserve all the RB properties.
- The properties that may be violated depends on the color of the deleted node.
  - Red – OK. Why?
  - Black?
- Steps:
  - Do regular BST deletion.
  - Fix any violations of RB properties that may result.

Deletion

RB-Delete(*T, z*)

1. **if** $left[z] = nil[T]$ or $right[z] = nil[T]$
2. **then** $y \leftarrow z$
3. **else** $y \leftarrow$ TREE-SUCCESSOR(z)
4. **if** $left[y] = nil[T]$
5. **then** $x \leftarrow left[y]$
6. **else** $x \leftarrow right[y]$
7. $p[x] \leftarrow p[y]$ //Do this, even if x is $nil[T]$

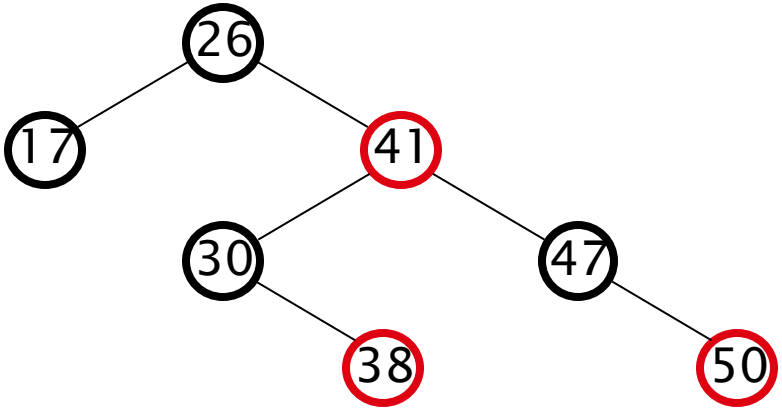
Deletion

RB-Delete (*T, z*) (Contd.)

```
8.  if p[y] = nil[T ]
9.      then root[T ] ← x
10.     else if y = left[p[y]]
11.             then left[p[y]] ← x
12.             else right[p[y]] ← x
13. if y = z
14.     then key[z] ← key[y]
15.     copy y's satellite data into z
16. if color[y] = BLACK
17.      then RB-Delete-Fixup(T, x)
18. return y
```

The node passed to the fixup routine is the lone child of the spliced up node, or the sentinel.

DELETE operation

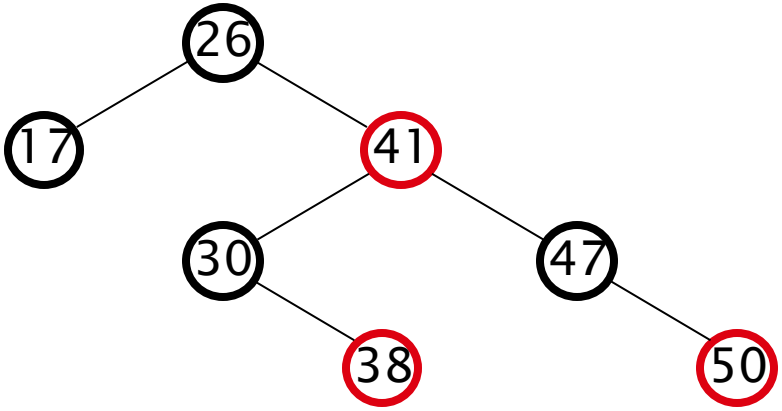


DELETE: the color of the node to be removed -- **red**

1. Every node is either **red** or **black** OK!
2. The root is **black** OK!
3. Every leaf (NIL) is **black** OK!
4. If a node is **red**, then both its children are **black** OK!
5. For each node, all paths from the node to descendant leaves contain the same number of **black** nodes OK!

Note: the deletion of a red node is the same as the deletion of a node in BST

DELETE operation



DELETE: the color of the node to be removed -- **Black**

1. Every node is either **red** or **black** OK!
2. The root is **black**

Not OK! If removing the root and the child that replaces it is **red**

3. Every leaf (NIL) is **black** OK!
4. If a node is **red**, then both its children are **black**

Not OK! Could create two red nodes in a row

Not OK! Could change the black heights of some nodes

5. For each node, all paths from the node to descendant leaves contain the same number of **black** nodes

Deletion on red-black tree

- Similar to the deletion on BST, but need to use an auxiliary procedure RB-Delete-Fixup to restore the red-black tree properties

- Four different cases of RB-Delete-Fixup
  - Case 1: x's sibling w is **red**
  - Case 2:x's sibling w is **black**, and both of w's children are **black**
  - Case 3:x's sibling w is **black**, w's left child is **red**, and w's right child is **black**
  - Case 4: x's sibling w is **black**, and w's right child is **red** (left child either color)

Deletion – Fixup

RB-Delete-Fixup(*T, x*)

```
1.  while x ≠ root[T] and color[x] = BLACK
2.      do if x = left[p[x]]
3.          then w ← right[p[x]]
4.               if color[w] = RED
5.                  then color[w] ← BLACK              // Case 1
6.                       color[p[x]] ← RED             // Case 1
7.                       LEFT-ROTATE(T, p[x])          // Case 1
8.                  w ← right[p[x]]                    // Case 1
```

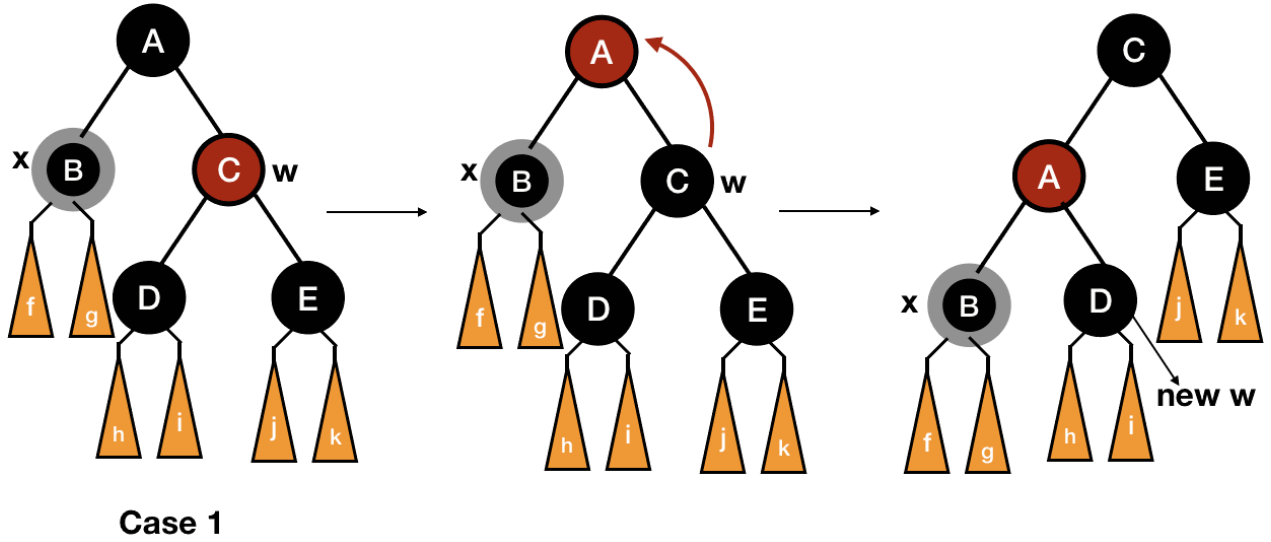
RB-Delete-Fixup(*T, x*) (Contd.)

```
        /* x is still left[p[x]] */
9.          if color[left[w]] = BLACK and color[right[w]] = BLACK
10.             then color[w] ← RED                                  // Case 2
11.                  x ← p[x]                                        // Case 2
12.             else if color[right[w]] = BLACK
13.                     then color[left[w]] ← BLACK                  // Case 3
14.                          color[w] ← RED                          // Case 3
15.                          RIGHT-ROTATE(T,w)                       // Case 3
16.                     w ← right[p[x]]                              // Case 3
17.                  color[w] ← color[p[x]]                          // Case 4
18.                  color[p[x]] ← BLACK                             // Case 4
19.                  color[right[w]] ← BLACK                         // Case 4
20.                  LEFT-ROTATE(T, p[x])                            // Case 4
21.                  x ← root[T]                                     // Case 4
22.     else (same as then clause with "right" and "left" exchanged)
23. color[x] ← BLACK
```

Deletion – Fixup

- ***Idea:*** Move the extra black up the tree until x points to a red & black node $\Rightarrow$ turn it into a black node,
- x points to the root $\Rightarrow$ just remove the extra black, or
- We can do certain rotations and recolorings and finish.
- Within the **while** loop:
  - x always points to a nonroot doubly black node.
  - w is x's sibling.
  - w cannot be $nil[T]$, since that would violate property 5 at $p[x]$.
- 8 cases in all, 4 of which are symmetric to the other.

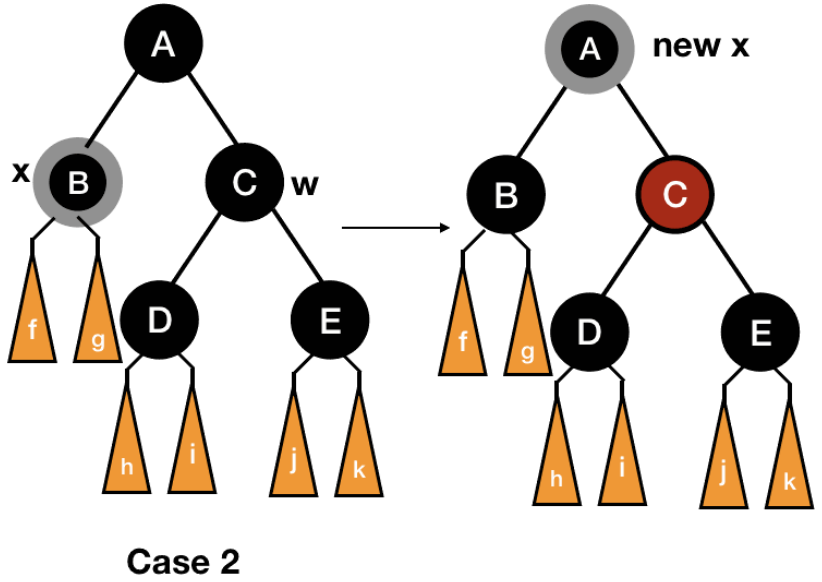
Case 1 – w is red



Case 1

- w must have black children.
- Make w black and $p[x]$ red (because w is red $p[x]$ couldn't have been red).
- Then left rotate on $p[x]$.
- New sibling of x was a child of w before rotation $\Rightarrow$ must be black.
- Go immediately to case 2, 3, or 4.

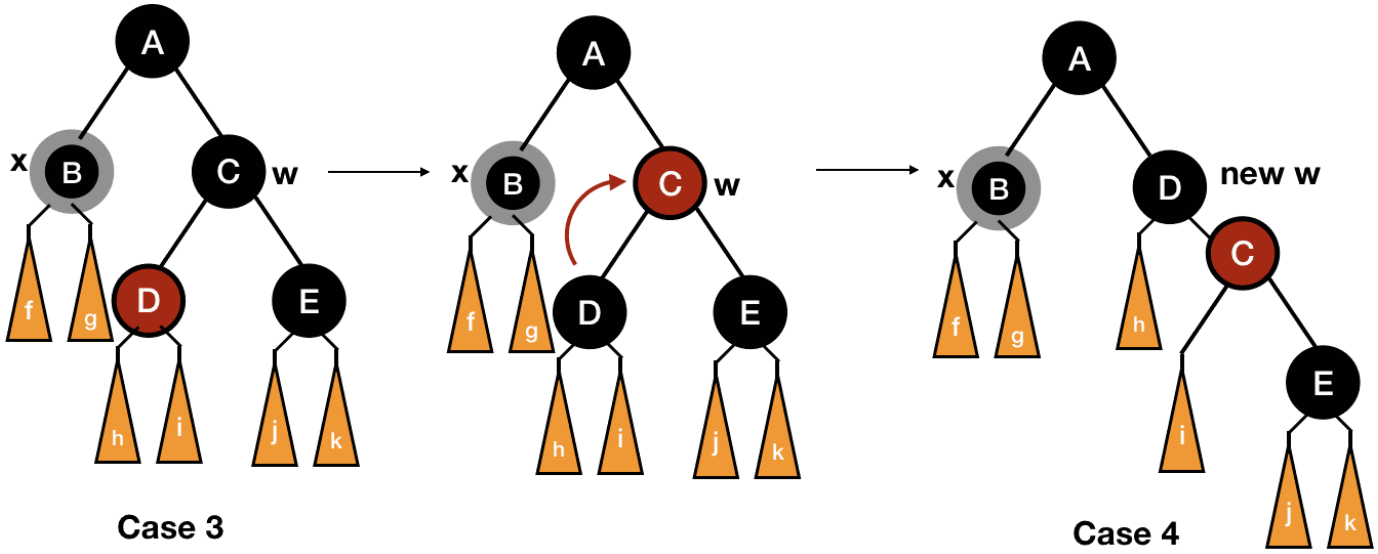
Case 2 – w is black, both w's children are black



Case 2

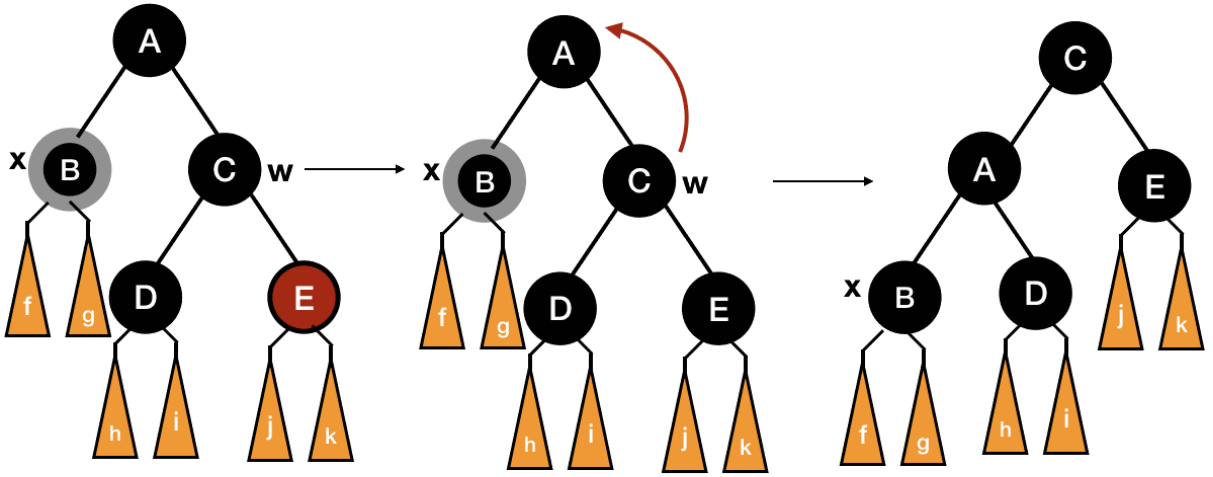
- Take 1 black off x ($\Rightarrow$ singly black) and off w ($\Rightarrow$ red).
- Move that black to $p[x]$.
- Do the next iteration with $p[x]$ as the new x.
- If entered this case from case 1, then $p[x]$ was red $\Rightarrow$ new x is red & black $\Rightarrow$ color attribute of new x is RED $\Rightarrow$ loop terminates. Then new x is made black in the last line.

Case 3 – *w* is black, *w*'s left child is red, *w*'s right child is black



- Make *w* red and *w*'s left child black.
- Then right rotate on *w*.
- New sibling *w* of *x* is black with a red right child ⇒ case 4.

Case 4 – *w* is black, *w*'s right child is red



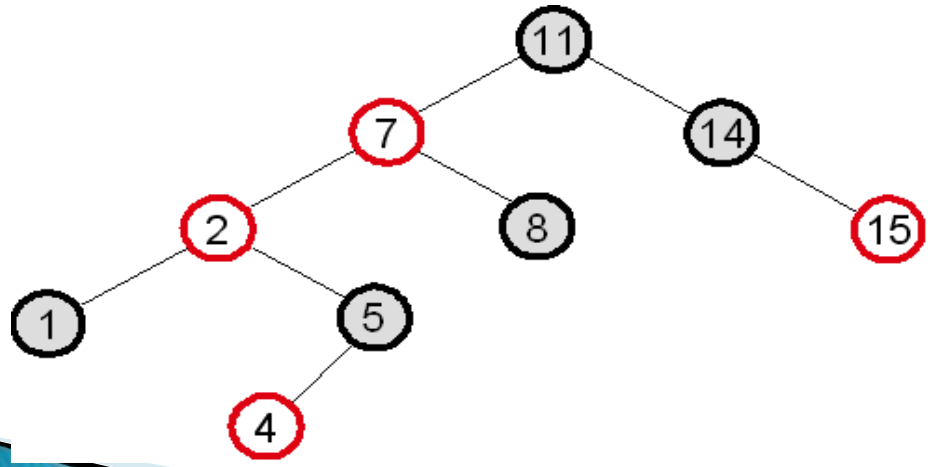
- Make *w* be *p*[*x*]'s color (*c*).
- Make *p*[*x*] black and *w*'s right child black.
- Then left rotate on *p*[*x*].
- Remove the extra black on *x* ($\Rightarrow$ *x* is now singly black) without violating any red-black properties.
- All done. Setting *x* to root causes the loop to terminate.

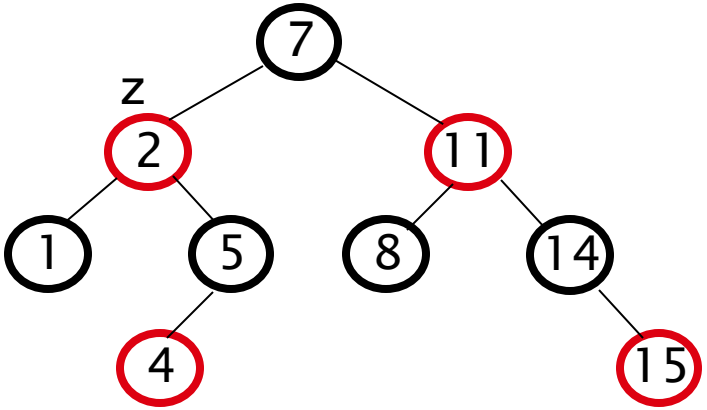
Analysis

- $O(\lg n)$ time to get through RB-Delete up to the call of RB-Delete-Fixup.
- Within RB-Delete-Fixup:
  - Case 2 is the only case in which more iterations occur.
    - x moves up 1 level.
    - Hence, $O(\lg n)$ iterations.
  - Each of cases 1, 3, and 4 has 1 rotation $\Rightarrow \leq 3$ rotations in all.
  - Hence, $O(\lg n)$ time.

Exercise 2

- What red-black tree property is violated in the tree below? How would you restore the red-black tree property in this case?
  - Property violated: if a node is red, both its children are black
  - Fixup: color 7 black, 11 red, then right-rotate around 11





Red-black trees: summary

- Red-black trees guarantee that the height of the tree will be O(logn)

- Operations on red-black-trees:
  - SEARCH O(h)
  - PREDECESSOR O(h)
  - SUCCESOR O(h)
  - MINIMUM O(h)
  - MAXIMUM O(h)
  - INSERT O(h)
  - DELETE O(h)